Born2beRoo

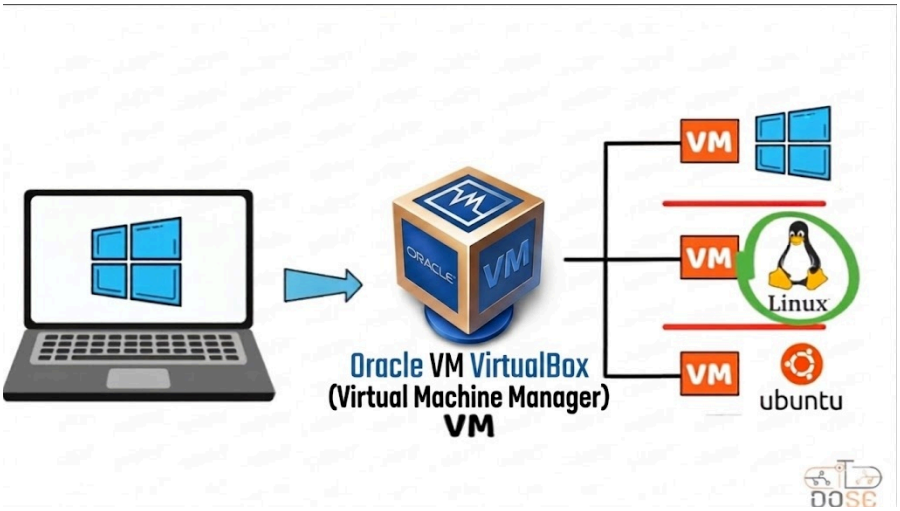
seba alawneh

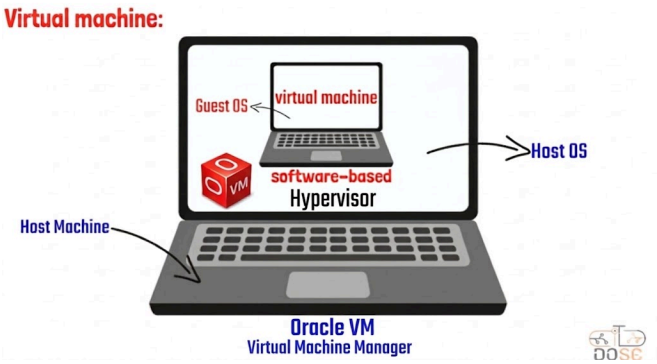
Virtualization

is a technology that enables the creation of several virtual environments with different operating systems on the same device.
One device, multiple environments: share the hardware of the same device

How do we create multiple environments?

Using the virtual machine (Oracle VirtualBox): it is software that allows us to create more than one virtual environment (VM). We also call it a hypervisor.





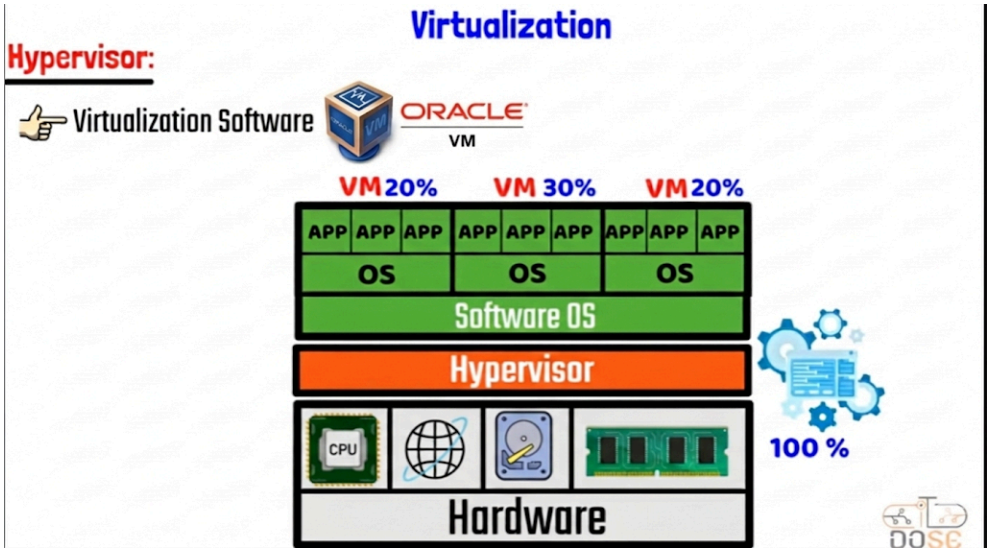
host machine: your physical device

host os : the os that you use in your device

virtual machine: is the new virtual “device” in the software that allows

you to create a virtual machine

Guest OS: is the os in your virtual device



Hypervisor: The hypervisor acts as a layer between the hardware and the virtual machines. It works as a manager that handles physical resources to customize them for each **virtual machine, distributing CPU, RAM, and storage to ensure they run independently and securely.**

VirtualBox vs UTM

In this project, I had to choose between two hypervisors based on my hardware architecture:

- **VirtualBox**:
  - **Architecture**: Designed for x86/amd64 systems (Intel and AMD processors).
  - **Availability**: Available on Windows, Linux, and Intel-based Macs
- **UTM**:
  - **Architecture**: Optimized for Apple Silicon (M1/M2/M3) using ARM architecture.

Choose the operating system

This activity consists of having you set up your first server by following specific rules.

Since it is a matter of setting up a server, you will install the minimum of services. For this reason, a graphical interface is of no use here. It is therefore forbidden to install `X.org`, or Wayland, or any other equivalent graphics server. Otherwise, your grade will be 0.

You must choose as an operating system either the latest stable version of `Debian` (no testing/unstable), or the latest stable version of `Rocky`. `Debian` is highly recommended if you are new to system administration.

First, as we see in the file, we have to choose one of two operating systems (Rocky / Debian).
Whatever you choose, you have to install the minimum of services.

> 💡 A minimal system has a smaller codebase. Fewer programs mean fewer bugs, fewer background processes to crash, and a much lower chance of library conflicts during updates.

WHY (minimum of services): with no graphical interface

Resource Efficiency: A Graphical User Interface (GUI) like X.org or Wayland is "heavy" and consumes significant **RAM** and **CPU** power just to display windows and icons. By sticking to the **minimum of services** (CLI only), the server can dedicate all its hardware resources to its actual purpose.

Security (Reducing the Attack Surface): Each extra service or package installed is a potential "door" for a hacker. By installing only the absolute minimum, we reduce The possibility of an attack, making the system much more secure and harder to exploit.

System Stability: A minimal system has fewer software conflicts and moving parts. This makes the server more stable, easier to update, and less likely to crash compared to a system loaded with unnecessary graphical tools.

Why Dabian?

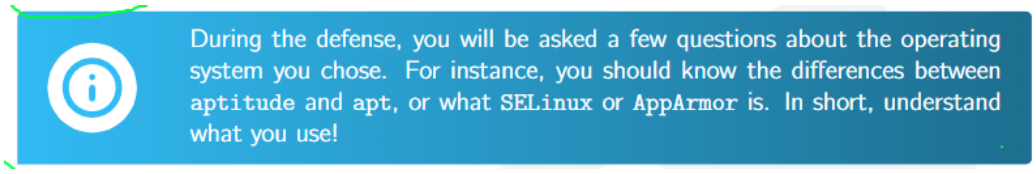
1) Recommended for Beginners: As per the project subject, Debian is highly recommended for those new to system administration

2) **Rock-solid Stability**: Debian is world-renowned for its stability and reliability in server environments.
3) **User-friendly Security**: It uses **AppArmor** by default, which is more intuitive to configure than Rocky Linux's **SELinux**.

> **How to verify AppArmor status:**
> To ensure the server is protected, run the command:
> `sudo aa-status`
> This command confirms that the AppArmor module is loaded and shows which profiles are in **Enforce mode** (actively blocking threats) versus **Complain mode** (only logging threats).

the differences between aptitude and apt

Know the difference between apt and aptitude! | Packagecloud Blog
Linux systems have essential concepts for installing software such as packages (software), and repositories. You need a package management s
https://blog.packagecloud.io/know-the-difference-between-apt-and-aptitude/



Package Management: apt vs. aptitude

- **What is apt?** It stands for **Advanced Package Tool**.
- **Role**: Both are p**ackage managers** responsible for installing, updating,

Key Differences:

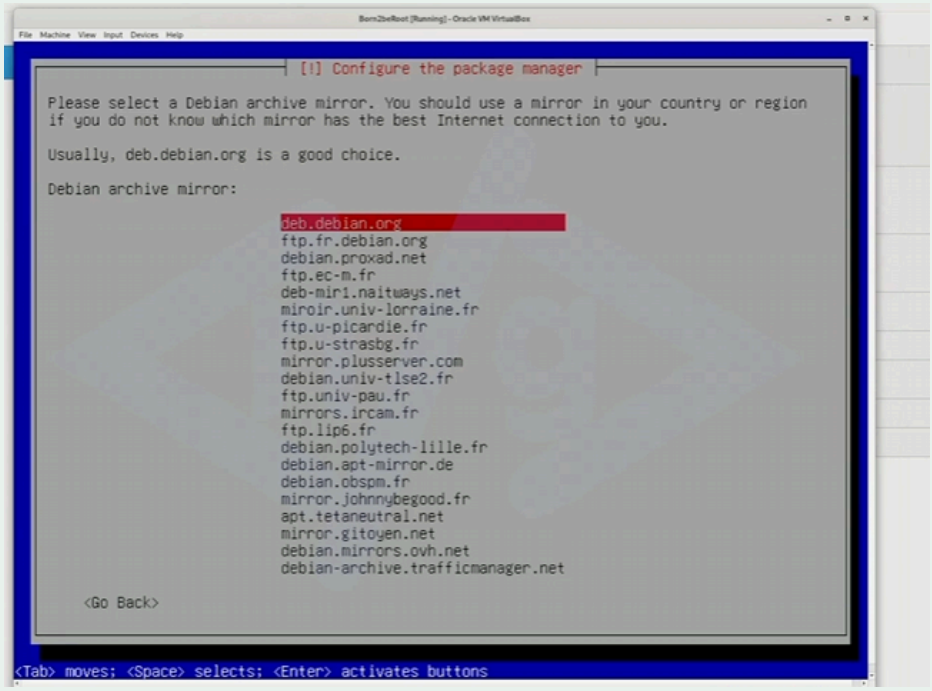
Feature	apt	aptitude
Interface	Pure **Command Line (CLI)** tool.	Has a **visual menu (TUI)** that you can navigate.
Conflict Handling	If a package conflicts with another, it shows an **error and stops**.	It is "smarter"; it **suggests solutions** and fixes to resolve conflicts.
Dependencies	Removes packages but might leave unused files behind (unless using `autoremove`).	**Automatically cleans up** unused dependencies when a package is removed.

and removing software on the system.

- **Project Examples**: We used them to install essential tools like
  - `sudo` (for administrative privileges).
  - `ufw` (the firewall).
  - `openssh-server` (for remote connection).

💡

- When you ask `apt` to install something, it fetches the packages from r**epositories** hosted on servers called m**irrors**.
- We define these "sources" during the **Debian installation** phase, specifically in the "Configure the package manager" step
- in this step:

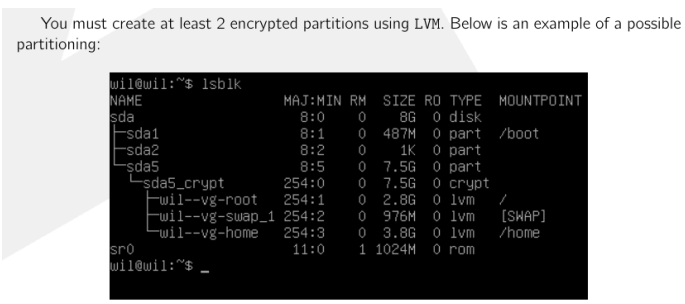


LVM

LVM (Logical Volume Manager) is a storage management technology that allows for flexible disk partitioning. Unlike physical partitioning, LVM allows us to **resize** partitions (logical volumes) easily without data loss. It is crucial to leave **unallocated space** in the Volume Group (VG) to allow for future expansion when a partition gets full

> LVM - ArchWiki
>
> Logical Volume Management utilizes the kernel's device-mapper feature to provide a system of partitions independent of underlying disk layout. With LVM you abstract your storage and have "virtual partitions", making extending/shrinking easier (subject to potential filesystem limitations).
>
> https://wiki.archlinux.org/title/LVM

What is this?



1️⃣ Hard Disk

The physical disk of the system.

sda

This represents the **entire hard drive**.

2️⃣ Partitions

The hard disk is divided into partitions.

sda1
sda2
sda5

Example in the project:

sda1 → /boot
sda5 → used for encryption and LVM

Partitions are similar to **C: and D:** drives in **Microsoft Windows**.

3️⃣ Encryption (LUKS)

The partition is encrypted using **LUKS**.

sda5

After unlocking the encrypted partition, it becomes:

sda5_crypt

Meaning:

sda5 = encrypted partition
sda5_crypt = unlocked encrypted partition

4️⃣ PV (Physical Volume)

The storage device used by LVM.

In this project:

PV = sda5_crypt

5️⃣ VG (Volume Group)

A storage pool created from one or more PVs.

VG = wil-vg

A storage pool created from one or more physical volumes.

Logical volumes are created from the available space in the volume group, and the data may be stored across multiple PVs.

6️⃣ LV (Logical Volumes)

Virtual partitions created inside the VG.

Wil-vg is the name of the VG

wil-vg-root → /
wil-vg-home → /home
wil-vg-swap_1 → swap

```
sda
└─ sda5
    └─ sda5_crypt  (PV)
         └─ wil-vg (VG)
              ├─ wil-vg-root
              ├─ wil-vg-home
              └─ wil-vg-swap_1
```

Installing sudo

1. What is sudo?

- **Definition**: "`sudo`" stands for **"Super User DO**."
- **Purpose**: It is a program that grants authorized users the ability to execute administrative tasks—such as installing software or modifying system configurations—as if they were the `root` user.

2. Why do we avoid using the root account directly?

- **The Danger Zone**: The `root` user is the most powerful account in the entire system.
- **High Risk**: Logging in as `root` directly is dangerous because any mistake or accidental command can lead to catastrophic system failure.

3. The "Sudo" Approach for Security

- **Principle of Least Privilege**: Instead of using `root`, we create a standard user (your login name) and grant it `sudo` rights.
- **The Safety Wall**: When using `sudo`, the system asks for a password before executing the command. This extra step forces the user to be intentional and "think twice" before making critical changes to the system.

creating a group

The Purpose of Groups 👥

- **User Categorization**: We create groups to organize users into specific sets based on their roles.

- **Centralized Permissions**: Instead of assigning permissions to every user individually, we assign them to a **Group** once (e.g., Permissions 1, 2, 3).
- **Efficient Management**: Any user added to that group automatically inherits all its predefined permissions, making system administration faster and more secure.

This user has to belong to the user42 and sudo groups.

> **SSH (Secure Shell) Overview:**
>
> - A protocol for secure remote access to the server.
> - It encrypts all communication to prevent data interception.
> - It uses a specific logical "port" to communicate. In this project, we changed the default port **22** to a custom port (**4242**) to enhance security by obscuring the service from automated attacks.
> - For SSH to work, the specific port must be allowed through the **UFW** (Uncomplicated Firewall).

Firewall

Firewall Comparison: UFW vs. firewalld
Both **UFW** and **firewalld** are firewall management tools. Their primary role is to monitor and control incoming and outgoing network traffic based on specific security rules, acting as a barrier to protect the server.

I chose **UFW** (Uncomplicated Firewall) for this project because it is characterized by its simplicity and ease of configuration, which reduces the possibility of setup errors. It was selected specifically because it is the native and recommended firewall for **Debian**, the operating system I used for this server.

In contrast, **firewalld** is the default firewall for systems like **Rocky Linux**. Its characteristics include a "zones" system, which allows for more advanced and dynamic network management but makes it significantly more complex to configure for minimal server setups.